到达计算机：Shell、MCP 和浏览器

Lab04 · 2026 年 10 月 12–13 日

layout: default
class: v2

今天的日程

第一部分：shell
第二部分：MCP
第三部分：浏览器
第四部分：提示注入
问题，然后是出勤表格

今天只有幻灯片：不需要安装任何东西，也不需要交任何东西。

layout: section

第一部分：Shell

它如何运行 · 它花费什么 · 它能到达什么

layout: section

短命令

第一部分 · shell

layout: default
class: v2

Shell · 短命令

一个命令，端到端

模型只询问。框架在你的计算机上运行命令并发送回它打印的内容。

<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>

每个框架中的相同循环，从最大的到 100 行的一个。

layout: default
class: v2

Shell · 短命令

Shell 工具的输入

模型写的不仅仅是命令：一个给你的描述，以及等待多长时间。

```
{"name": "Bash",
 "input": {
   "command": "pytest -q",
   "description": "Run the test suite",
   "timeout": 120000
 }}
```

`command`	运行什么
`description`	询问你批准时你看到的内容

layout: default
class: v2

Shell · 短命令

一次一个工具调用

通常每个命令是它自己的工具调用。每个结果在模型选择下一个命令之前返回给它。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
一个说明性示例：shell() 代表框架的 shell 工具。

layout: default
class: v2

Shell · 短命令

Code 模式：从代码调用的工具

在 code 模式中，模型的工具调用是一个程序。框架运行整个程序，一个结果返回：程序打印的任何内容。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
一个说明性示例：shell() 代表框架的 shell 工具，从 Python 调用。Anthropic 的"Code execution with MCP"（2025 年 11 月 4 日）和 Cloudflare 的"Code Mode"（2025 年 9 月 26 日）描述了同样的想法。

layout: default
class: v2

Shell · 短命令

在同样的车道中 Code 模式

模型被调用一次。命令在它等待时运行，只有打印的结果到达它的上下文。
<v-switch> <template #0></template> <template #1></template> <template #2></template> <template #3></template> </v-switch>
与最后两张幻灯片同样的说明性示例。

layout: section

长时间运行的命令

第一部分 · shell

layout: default
class: v2

Shell · 长时间运行的命令

不会结束的命令

从外部，慢命令和永不结束的命令看起来一样。框架必须决定何时停止等待。

服务器

`npm run dev`, `python -m taskrweb`

监视器

`tsc --watch`, 任何在保存时重新运行的东西

等待输入

密码提示、分页器、是或否

layout: default
class: v2

Shell · 长时间运行的命令

等待输入的命令

默认情况下没有人在键盘前，所以框架阻止程序停下来询问。

程序想要	框架做什么
一个分页器（`git log`）	设置 `PAGER=cat`，所以它打印并退出
颜色和进度条	设置 `TERM=dumb` 和 `NO_COLOR=1`
一个编辑器或提示（`git rebase -i`）	告诉模型不要运行它们

Codex 0.161 源代码：每个命令得到 TERM=dumb, NO_COLOR=1 和 PAGER=cat · Claude Code 2.1.293 Bash 工具描述：不支持交互式标志。

layout: default
class: v2

Shell · 长时间运行的命令

等待有截止日期

框架等待最多 2 分钟。然后它停止等待，但命令继续运行，框架在它结束时告诉模型。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
从 Claude Code 2.1.293 的 Bash 工具简化（默认超时 2 分钟，最多 10 分钟），10 月 7 日捕获，及其文档。

layout: default
class: v2

Shell · 长时间运行的命令

在后台启动

当模型知道一个命令不会结束时，它 upfront 说出来。框架立即用 task id 回答。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
从 Claude Code 2.1.293 的 Bash 工具简化（run_in_background），10 月 7 日捕获。

layout: default
class: v2

Shell · 长时间运行的命令

进入运行中命令的四种方式

一个运行中的命令有输出、输入、端口和停止。模型通过框架到达每一个，通过 task id。
<v-switch> <template #0></template> <template #1></template> <template #2></template> <template #3></template> <template #4></template> </v-switch>
从 Claude Code 2.1.293 的 Monitor 和 TaskStop 以及 Codex 0.160.0 的 write_stdin 简化，10 月 7 日捕获。

layout: section

第二部分：MCP

其他人写的工具

layout: default
class: v2

MCP · 它是什么

MCP 是什么

MCP 是框架如何从其他程序获得更多工具，并将它们交给模型。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
MCP，Model Context Protocol，是一个开放标准（modelcontextprotocol.io），由 Anthropic 在 2024 年 11 月宣布，现在在 Linux Foundation 下。

layout: default
class: v2

MCP · 为什么一个标准

没有 MCP，每个框架写自己的

Codex、Claude Code 和 ChatGPT 都会写自己的 GitHub 代码，以及他们自己的 Slack 代码，等等。



四个框架和四个服务：十六个集成，每个由框架的制作者编写和维护。

layout: default
class: v2

MCP · 为什么一个标准

有了 MCP，服务写一次

GitHub 写一个 MCP 服务器。每个会说 MCP 的框架都可以使用它。



每个框架说一次 MCP，每个服务写一个服务器：八块而不是十六块。这个想法是从 Language Server Protocol 借来的。

layout: default
class: v2

MCP · 它如何运行

MCP 在两个调用中

框架询问服务器它的工具，然后调用一个。模型从不与它对话。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
两个调用是 MCP 规范中的 tools/list 和 tools/call。工具名称来自 GitHub 的 MCP 服务器；调用和结果是说明性的。

layout: default
class: v2

MCP · 它如何运行

服务器告诉模型什么

对于每个工具：一个名称，何时使用它的描述，以及它需要的参数。

```
{"name": "list_issues",
 "description": "List issues in a GitHub repository with optional
                 filtering by labels, states, and custom fields.",
 "inputSchema": {"type": "object",
   "properties": {
     "owner": {"type": "string", "description": "The owner of the repository"},
     "repo":  {"type": "string", "description": "The name of the repository"},
     "state": {"type": "string", "description": "Filter by issue state (open, closed, all)"}},
   "required": ["owner", "repo"]},
 "annotations": {"readOnlyHint": true}}
```

来自 GitHub MCP 服务器源（github/github-mcp-server）的 list_issues，修剪到它的

layout: default
class: v2

MCP · 它如何运行

MCP 服务器运行在哪里

一个本地服务器是框架在你的计算机上启动的程序。一个远程服务器在其他人的机器上运行。

<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>

MCP 规范，2026-07-28 · modelcontextprotocol.io

layout: default
class: v2

MCP · 它如何运行

框架如何发送一个调用

一行 JSON。一个本地服务器从它的输入读取它；一个远程服务器通过 HTTPS 接收它。

```
the harness writes:
{"jsonrpc": "2.0", "id": 3, "method": "tools/call",
 "params": {"name": "list_issues",
            "arguments": {"owner": "Aider-AI", "repo": "aider"}}}
the server answers:
{"jsonrpc": "2.0", "id": 3,
 "result": {"content": [{"type": "text", "text": "[{\"number\": 4521, …}]"}]}}
```

MCP 规范：JSON-RPC 2.0，本地服务器的 stdin 和 stdout 上每行一个消息。修剪；结果是说明性的。

layout: default
class: v2

MCP · 不止工具调用

不止工具调用

服务器还可以处理登录、共享数据、问你问题，以及添加工具。

登录

登录一次；token 留在服务器中

数据

文件和记录，不只是工具

问题

问你它缺少的东西

layout: default
class: v2

MCP · 它花费什么

MCP 服务器或 CLI

如果框架有一个 shell，模型已经知道的 CLI 通常更便宜。

gh 通过 shell
模型已经知道它

每个请求不添加任何东西

需要 shell 和你的登录

GitHub MCP 服务器
类型化参数，描述的工具

layout: default
class: v2

MCP · 它花费什么

按需加载工具

框架只向模型发送工具名称。当模型需要一个工具时，它搜索，框架加载那一个定义。
<v-switch> <template #0></template> <template #1></template> <template #2></template> <template #3></template> </v-switch>
Playwright MCP 0.0.83 的 25 个工具，10 月 7 日捕获，o200k token · Anthropic 测量 77K token 降到 8.7K（2025 年 11 月 24 日）。

layout: section

第三部分：浏览器

页面、像素和树

layout: default
class: v2

Browser · 它如何运行

三种方式看一个页面

屏幕截图显示布局，树命名什么可以点击，DOM 保存一切。

像素：屏幕截图



可访问性树

```
- navigation "Main" [ref=e7]:
  - list [ref=e8]:
    - listitem [ref=e9]:
      - link "Installation" [ref=e13]
```

DOM，通过 JavaScript

layout: default
class: v2

Browser · 它如何运行

通过引用点击

模型点击树中的一个名称，不是像素，回复说它点击了什么。

<v-switch> <template #0>

```
- navigation "Main" [ref=e7]:
  - list [ref=e8]:
    - listitem [ref=e9]:
      - link "Installation" [ref=e13]
```



</template> <template #1>

```
      - link "Installation" [ref=e13]
```

layout: default
class: v2

Browser · 它如何运行

浏览器工具，并排

Chrome 中的 Claude 工作像 Playwright：同样的三个视图，并通过引用点击。

	Playwright MCP	Chrome DevTools MCP	Chrome 中的 Claude
屏幕截图	`browser_take_screenshot`	`take_screenshot`	`computer` screenshot
树	`browser_snapshot`	`take_snapshot`	`read_page`
DOM	`browser_evaluate`	`evaluate_script`	`javascript_tool`

layout: default
class: v2

Browser · 它花费什么

一个页面，两种成本

在这个页面上，树的成本几乎是屏幕截图的六倍。哪个更好用取决于页面。
1,196
1280 × 720 屏幕截图的 token
6,802
同样页面的树的 token，523 行
屏幕截图：Anthropic 的 28 × 28 像素块。树：o200k tokenizer ◦WebVoyager（ACL 2024）：在 Booking 和 Google Flights 上，屏幕截图以 41 和 52 分领先于树。

layout: default
class: v2

Browser · 它能到达什么

进入浏览器的三种方式

每向外一步到达更多的你：一个 fresh 浏览器，你登录的 Chrome，你的整个屏幕。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
Playwright MCP README · Chrome DevTools MCP（Google，2025 年 9 月）· Vibium（J. Huggins）· Chrome 中 Claude 文档 · Anthropic 计算机使用文档。

layout: section

第四部分：提示注入

当模型读取的内容给出命令

layout: default
class: v2

Prompt injection · 它是什么

什么是提示注入

提示注入 是模型读取的文本，它*跟随它，就像你写的*。

通过 shell
你克隆的仓库中的 README 或文件

通过 MCP
工具的描述，或它获取的问题

通过浏览器
它访问的任何页面上的隐藏文本

layout: default
class: v2

Prompt injection · 通过 MCP

服务器在你提示中的话

工具描述是你的模型的一个指令。安装一个像代码一样作为你运行的服务器。
<v-switch> <template #0></template> <template #1></template> <template #2></template> </v-switch>
在 Invariant Labs，"Tool Poisoning Attacks," 2025 年 4 月 1 日之后。MCP 规范：工具描述不受信任，除非服务器受信任。

layout: default
class: v2

Prompt injection · 所有三个通道

致命的三位一体

私有数据、不受信任的内容和一个发送数据出去的方式，在一个框架中，就是数据如何被盗。

<v-switch> <template #0></template> <template #1></template> <template #2></template> <template #3></template> </v-switch>

Simon Willison，"The lethal trifecta for AI agents," 2025 年 6 月 16 日。

layout: default
class: v2

Prompt injection · 每个通道

每个通道中的三位一体

每个通道可以自己供应所有三条腿。切断一条腿，攻击失败。
<v-switch> <template #0></template> <template #1></template> <template #2></template> <template #3></template> </v-switch>
MCP 行发生了：Invariant Labs，2025 年 5 月 26 日。公共仓库上的一个问题导致带有 GitHub MCP 服务器的框架将私有仓库数据复制到公共 pull 请求。

layout: end